

ACSOS · document-to-LLM

Architektur- und Entwicklerdokumentation — Extraktions-, Verifikations- und Publikations-Pipeline fuer lizenzierte GRC-Normen und Fachdokumente.

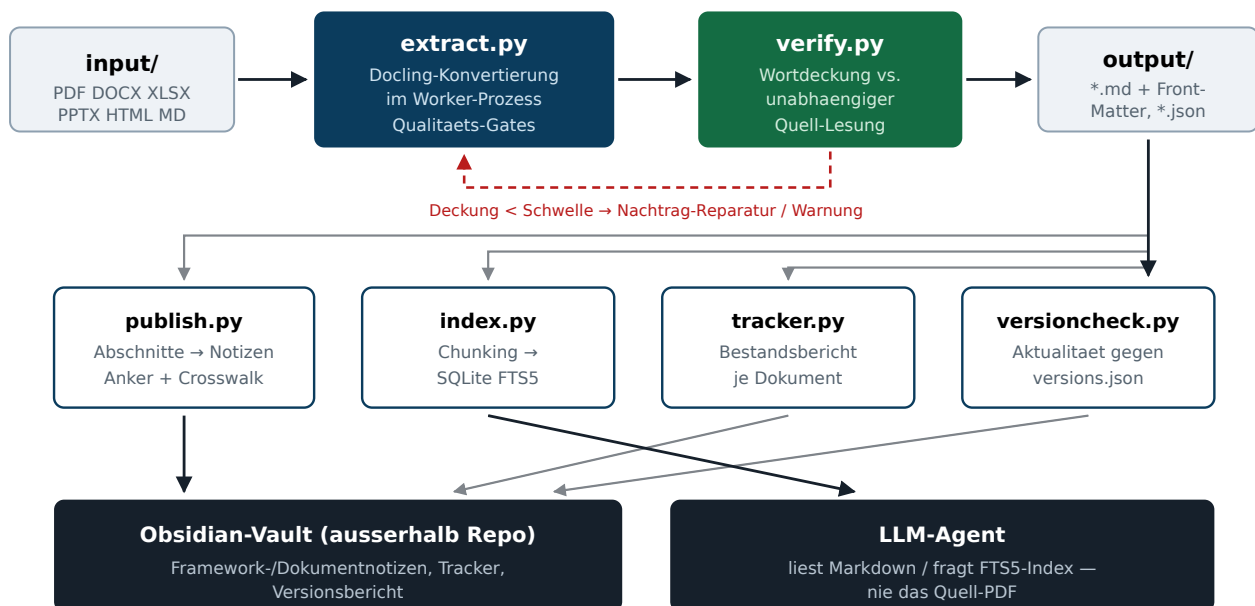
Kernregel der Architektur. Konvertiert wird ausschliesslich mit **IBM Docling**. Es existiert kein eigener PDF-Parser, kein PyPDF-, pdfminer- oder Regex-Pfad. Layout-, Tabellen- und Hierarchieerkennung machen die Docling-Layoutmodelle. Ein LLM-Agent liest **nie** ein PDF im Kontextfenster, sondern ausschliesslich das erzeugte Markdown aus `output/`.

1 Zweck und Systemgrenze

Die Pipeline nimmt lizenzierte Quelldokumente (PDF, DOCX, XLSX, PPTX, HTML, MD) entgegen und erzeugt daraus strukturerhaltendes Markdown mit vollstaendiger Provenienz. Anschliessend wird maschinell geprueft, dass der Zieltext den Quelltext **wortdeckend** abbildet; erst danach werden Inhalte in den Obsidian-Vault veroeffentlicht und in einen Retrieval-Index gestellt.

Systemgrenze: alles laeuft lokal als CLI. Keine Serverkomponente, kein Netzwerkzugriff zur Laufzeit ausser dem einmaligen Modell-Download von Hugging Face (danach `HF_HUB_OFFLINE=1`). Lizenziertes Normtext verlaesst die Maschine nicht und wird niemals versioniert.

2 Datenfluss



Durchgezogen = Hauptpfad, gestrichelt rot = Ruecklauf bei Deckungsunterschreitung, transparent = Nebenseiten auf denselben `output/`-Bestand.

3 Module

extract.py 802 LOC

Konvertierung. Baut den `DocumentConverter` mit `PdfPipelineOptions` (TableFormer, Cell-Matching, Layoutmodell), exportiert seitenweise Markdown mit `<!-- page: N -->`-Markern, schreibt YAML-Front-Matter und ruft die Qualitäts-Gates auf.

verify.py 405 LOC

Unabhängige Gegenlesung der Quelle: PDF über den Docling-Textlayer-Backend, Office-Formate über `openpyxl` / `python-docx` / `python-pptx` — bewusst **nicht** über denselben Pfad wie die Konvertierung, damit der Vergleich aussagekräftig ist.

publish.py 504 LOC

Schreibt Abschnitte in den Vault. Findet die Fundstelle einer Control-ID über ID-Varianten, Überschriften, Tabellenzeilen und Volltext-Anker; optionaler Crosswalk aus `mappings/*.json`. `--as-document` erzeugt stattdessen eine Dokumentnotiz.

index.py 306 LOC

Retrieval-Layer: zerlegt die Extrakte (HierarchicalChunker bzw. Überschriftenpfad) und legt sie in SQLite-FTS5. Ein Agent holt sich damit den einzelnen Absatz mit Dokument, Gliederungspfad und Seitenzahl als Beleg.

tracker.py 223 LOC

Erzeugt aus Front-Matter und Vault-Stand den Bestandsbericht je Dokument (Seiten, Tabellen, Überschriften, Wortdeckung, entfernte Kopf-/Fusszeilen) — in `output/_TRACKER.md` und in den Vault.

versioncheck.py 213 LOC

Prüft je Dokument die Aktualität gegen `versions.json`. Statuswerte: `aktuell` · `VERALTET` · `belegt` (aus Dokument, mit Datei und Seite) · `manuell` · `unerreichbar`.

4 Konvertierungspfad im Detail

4.1 Prozessisolation

Docling-Layoutmodelle koennen bei einzelnen Dokumenten hart abstuerzen (beobachtet: Signal 11 mitten in einem 14-Dokument-Lauf). Deshalb laeuft jede Konvertierung in einem `ProcessPoolExecutor(max_workers=1)`. Ein `BrokenProcessPool` wird abgefangen, der Pool zurueckgesetzt und nur **das eine** Dokument als Fehler markiert — der Batch laeuft weiter.

4.2 Modelle und Offline-Betrieb

Layout- und Tabellenmodelle kommen aus dem Hugging-Face-Cache (`docling-layout-heron`, `ds4sd/docling-models TableFormer`). Netzwerkfehler beim Modell-Download loesen einen automatischen Retry mit `HF_HUB_OFFLINE=1` gegen den lokalen Cache aus. `--models-dir` (bzw. `ACS0S_DOCLING_MODELS`) setzt `artifacts_path` fuer den vollstaendig luftdichten Betrieb.

4.3 Tabellenmodus mit Ruecknahme

`TableFormerMode.ACCURATE` ist nicht deterministisch. Bei einem gescheiterten Lauf wird `ACCURATE` einmal wiederholt, danach auf `FAST` zurueckgenommen; der tatsaechlich verwendete Modus steht als `table_mode` im Front-Matter samt Warnung.

4.4 Docling-Status auswerten

Erfahrung aus dem Betrieb: Ein ignoriertes `PARTIAL_SUCCESS` hat einmal eine stillschweigend abgeschnittene Datei erzeugt (fehlende Seite, verlorener Anhang, 60,9 % Deckung). Seitdem werden `result.status` und `result.errors` ausgewertet und in `docling_status` festgehalten.

5 Qualitaets-Gates

Gate	Prueft	Reaktion
Leeres Ergebnis	Markdown ohne Inhalt	Abbruch (Fehler)
Zeichen pro Seite	Auffaellig duenne Seiten → fehlender Textlayer / OCR noetig	Warnung
Ueberschriften	Dokument ohne jede Gliederung (uebersprungen fuer xlsx/xlsm/csv)	Warnung
Tabellenkonsistenz	Ungleiche Spaltenzahl innerhalb eines Tabellenblocks	Warnung
Cell-Spill	Zelle > 25 Zeichen, die klein beginnt → Text aus der Nachbarzelle uebergelaufen (Inhaltsverzeichnis-Zeilen mit Punktuehrung ausgenommen)	Warnung + Hinweis-Callout in der Vault-Notiz
U+FFFD	Ersetzungszeichen = kaputte Kodierung	Warnung
Wortdeckung	Anteil der Quellwoerter, die im Extrakt wiederzufinden sind, Schwelle <code>--min-coverage</code> (Standard 99,5 %)	Reparatur, sonst Warnung / <code>--strict</code> = Exit 1

5.1 Wie die Wortdeckung gemessen wird

- Quelle wird unabhaengig gelesen (Textlayer-Backend bzw. openpyxl/python-docx/python-pptx).
- Normalisierte Tokenisierung, Vergleich seitenweise gegen die Seitenmarker im Extrakt.
- Laufende Kopf-/Fusszeilen werden erkannt (Ziffern maskiert, Wiederholung ueber Seiten) und aus der Bezugsmenge genommen — sie fehlen im Extrakt **absichtlich**.
- Am Zeilenumbruch getrennte Woerter werden vor dem Vergleich wieder zusammengefuehrt.
- Was danach fehlt, haengt `--repair` als `##` Nachtrag: nicht zugeordneter Quelltext mit Seitenmarker und Zitatpraefix an — nichts geht verloren, und es ist sichtbar, dass es Nachtrag ist.

6 Datenvertrag: Front-Matter

Jedes Extrakt traegt seine eigene Provenienz. Der Tracker, `versioncheck.py` und `index.py` lesen ausschliesslich diese Felder — sie sind die stabile Schnittstelle zwischen den Modulen.

```
---
source_file:      ISO_IEC_27001_2022.pdf
source_sha256:    3f9c...           # Identitaet der Quelle
source_bytes:     1300551
pages:           31
tables:          12
converter:       docling 2.123.0
ocr:             false
table_mode:      accurate           # oder fast nach Ruecknahme
docling_status:  SUCCESS            # PARTIAL_SUCCESS wird nicht verschluckt
converted_at:    2026-08-27T09:14:02Z
text_coverage_percent: 100.0        # Ergebnis von verify.py
appended_source_lines: 0            # Umfang des Nachtrags
extraction_status: ok | warn | error
warnings:        []
---
```

7 Publikation in den Vault

Der Vault liegt bewusst **ausserhalb** des Repositories. `publish.py` loest eine Control-/Artikel-ID in der Reihenfolge auf:

- **ID-Varianten** — `A.8.1`, `A 8.1`, `8.1`, normalisiert ueber `norm_key()`.
- **Ueberschriften** — Abschnitt inkl. Aggregation der Elternebene.
- **Tabellenzeilen** — Control-Kataloge, die als Tabelle vorliegen.
- **Volltext-Anker** — Regex auf Zeilenanfang/Zellgrenze; wenn der Treffer kuerzer als 40 Zeichen ist, wird ueber die naechste Ueberschrift hinaus verlaengert (EU-Rechtsakte setzen „Artikel 7“ allein, der Titel folgt als naechste Zeile).
- **Crosswalk** — `mappings/*.json` fuer thematische statt positioneller Zuordnung; die Suche laeuft ueber die **vollstaendige** Phrase, nicht die ersten Woerter (sonst trifft „Produkte mit digitalen Elementen“ die Titelseite).

Lizenzgrenze. Weder `input/` noch `output/` werden versioniert (nur `.gitkeep`). Lizenziierter Normtext existiert ausschliesslich lokal und im Vault. Das gilt fuer beide Repositories.

8 Betrieb

```
# Vorflug: Modelle, Backends, Schreibrechte
python extract.py --doctor

# Batch mit Verifikation und Reparatur (Standard)
python extract.py input/ --recursive --json

# Einzelne Datei gegen ihre Quelle nachpruefen
python verify.py output/iso-27001.md --source input/ISO_27001.pdf --show-missing 25

# Retrieval-Index bauen und abfragen
python index.py build --output output --db output/acsos.db
python index.py search "Kryptographie Schluesselverwaltung" --db output/acsos.db

# Bestandsbericht und Aktualitaetspruefung
python tracker.py --output output --input input --vault "$VAULT" \
  --to output/_TRACKER.md --to "$VAULT/document-to-LLM Tracker.md"
python versioncheck.py --output output --to "$VAULT/document-to-LLM Versionen.md"

# In den Vault veroeffentlichen
python publish.py output/iso-27001.md --vault "$VAULT" --framework "ISO 27001"
```

8.1 Exit-Codes

Code	Bedeutung
0	Alle Dokumente konvertiert; ggf. Warnungen (ohne <code>--strict</code>)
1	Mindestens ein Fehler — oder mit <code>--strict</code> bereits eine Warnung; ebenso <code>versioncheck --strict</code> bei veralteter Version

9 Tests und CI

Workflow `.github/workflows/verify.yml` laeuft auf `push` (nur `main`), `pull_request` und `workflow_dispatch`, mit Concurrency-Gruppe je Ref und `cancel-in-progress`, damit parallele Pushes keine doppelten Laeufer und Mails erzeugen.

Stufe	Inhalt
Fixture-Generator	<code>tests/make_fixture.py</code> erzeugt die Testdokumente reproduzierbar — kein lizenzierter Text im Repo
Unit-Tests	<code>tests/test_units.py</code> , 35+ Pruefungen: Kopfzeilenerkennung, Zeilenzusammenfuehrung, Slug-Kollision, Anker-Aufloesung, Office-Verifikation
DOCX-Smoke	<code>tests/smoke.sh</code> — Pfad ohne Layoutmodelle
PDF-Smoke	<code>tests/smoke_pdf.sh</code> — voller Pfad inkl. Modell-Cache (HF-Cache wird in CI gecacht)
Artefakt	Extraktionsergebnis wird als Build-Artefakt hochgeladen

10 Erweiterungspunkte

- **Neues Quellformat:** von Docling unterstuetzte Formate laufen ohne Codeaenderung. Neu ist nur der **unabhaengige** Gegenleser in `verify.py::source_pages()` — genau der macht die Deckungszahl belastbar.
- **Neues Framework:** Crosswalk als `mappings/<framework>.json`; ohne Crosswalk greift die Anker-Aufloesung.
- **Neue Versionsquelle:** Eintrag in `versions.json`; ist online nichts erreichbar, belegt `from_document()` die Version aus dem Extrakt selbst (mit Datei und Seite als Nachweis).
- **Anderes Chunking:** `index.py --chunker hierarchical|hybrid|markdown`; `hybrid` braucht zusaetzlich `transformers`.

11 Bewusste Entwurfsentscheidungen

Entscheidung	Begrueundung
Nur Docling, kein eigener Parser	Eigene Regex-/PyPDF-Pfade verlieren Tabellenstruktur und Lesereihenfolge genau bei den Dokumenten, auf die es ankommt.
Gegenlesen mit anderer Bibliothek	Eine Deckungszahl aus derselben Engine wuerde nur bestaetigen, was die Engine ohnehin gelesen hat.
Ein Worker-Prozess je Dokument	Native Abstuerze der Layoutmodelle duerfen keinen Batch von Dutzenden Dokumenten toeten.
Nachtrag statt stiller Verlust	Unzuordenbarer Quelltext bleibt sichtbar erhalten — fuer eine Norm ist stiller Textverlust der teuerste Fehler.
Front-Matter als Modulschnittstelle	Alle nachgelagerten Werkzeuge lesen nur Dateien, keinen gemeinsamen Zustand — jeder Schritt ist einzeln wiederholbar.
Vault ausserhalb des Repos	Lizenzbindung des Normtextes; das Repo bleibt frei verteilbar.

